機械時計のしくみ

発表：2-2 斉藤 健吾

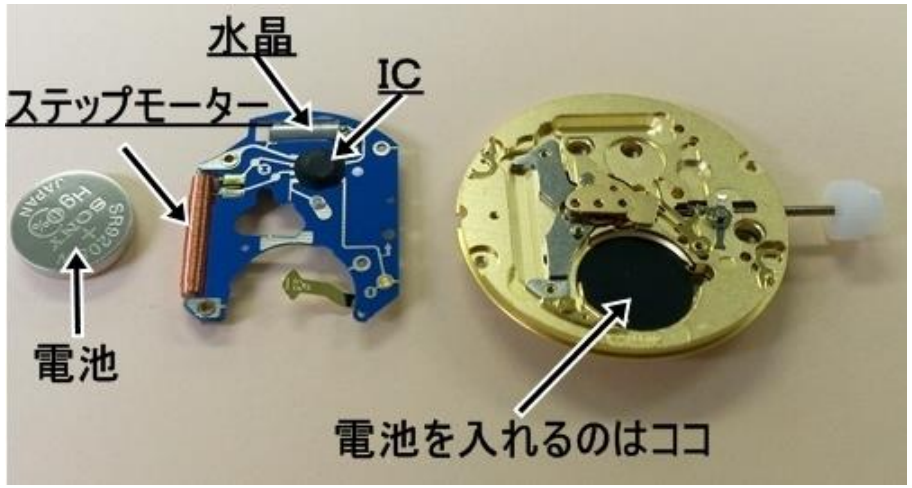
時計の種類

様々な時計

時計には様々な時計がある。

クォーツ時計、電波時計、光発電、機械式時計、
この中でも、機械式時計について研究しようと考えた。

クォーツ時計



https://www.hanshin-dept.jp/

機械式時計



https://www.rasin.co.jp/

電波時計



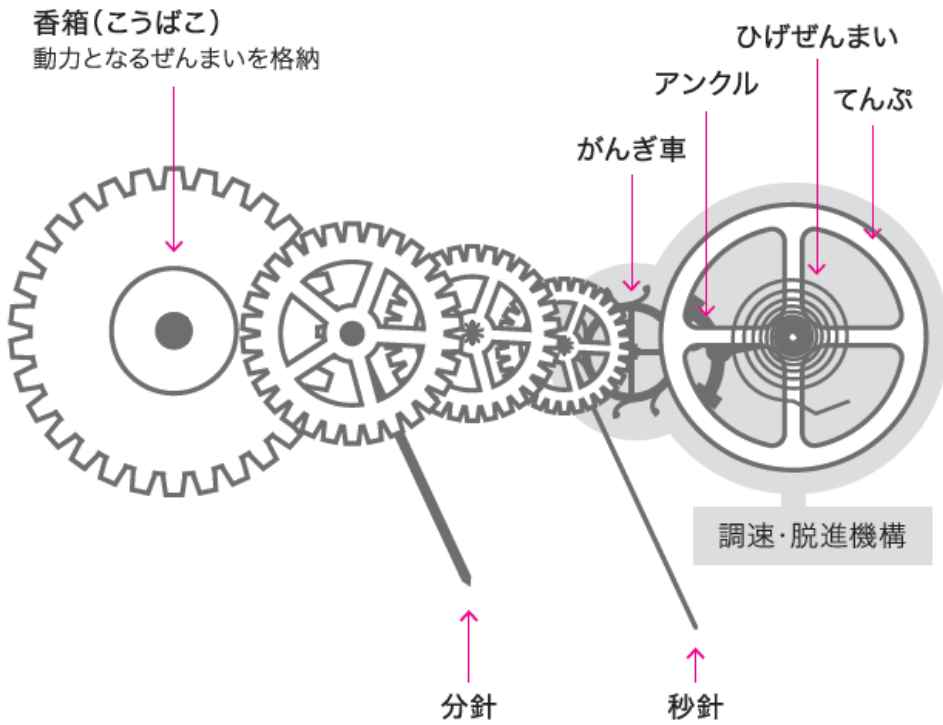
https://www.e-adesso.co.jp/

機械時計の仕組み

仕組み

機械時計は巻き上がったゼンマイがほどけようとする力で動く。 ⇒ゼンマイが切れたら動かなくなる。

ゼンマイの力を動力にして、てんぷの振動により、歯車を動かして針を動かす。



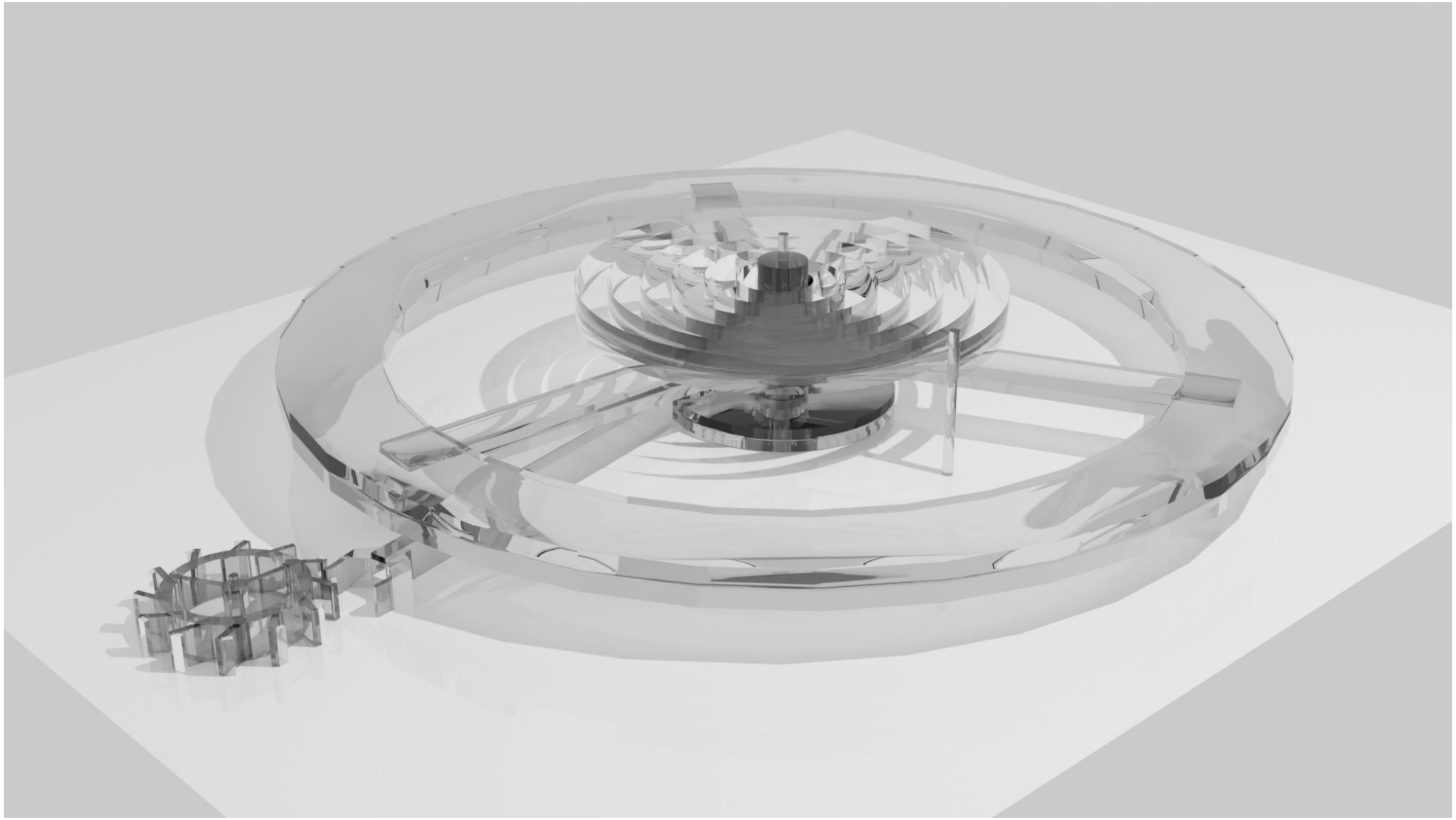
てんぷとは

右回りと左回りを繰り返しながら
同じ周期（1秒間に 5～10回、片道ベース）で
往復回転運動するように作られている部品である。

ひげせんまいとは

巻状に巻かれた細いひげぜんまいを内蔵するコマのような形をした部品。

モデリング(Blender)



コリジョン(当たり判定)



イメージ（理想）

Blenderで作って
Unityで動かそ。



コリジョン(当たり判定)

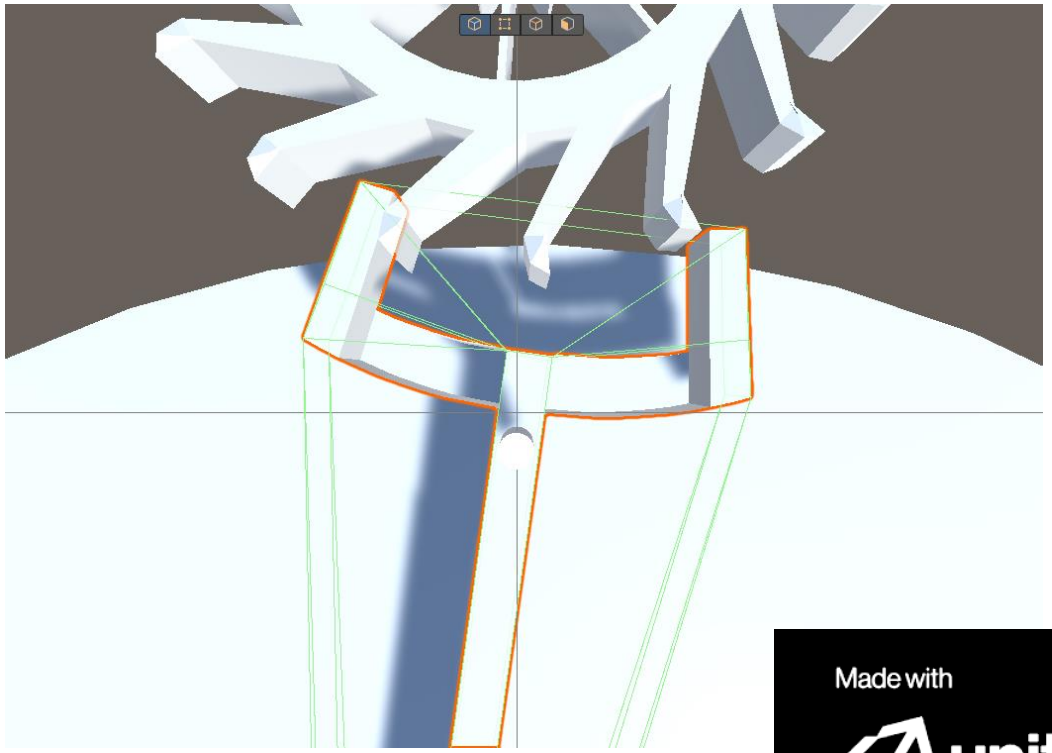


イメージ（理想）

Unityで動かしたら歯車が
かみ合うんでしょ知らんけど 。



コリジョン(当たり判定)



最小のコリジョンでも粗い。
調べてみてもどちらとも動力
をつけてるのばかり…



Made with unity

現実

解決策...

解決策1

一つ一つの歯に分けてコライダーを割り当て、
その小さな部品の集合を一つのオブジェクトとして扱う。



解決策...

解決策1

一つ一つの歯に分けてコライダーを割り当て、
その小さな部品の集合を一つのオブジェクトとして扱う。

→処理が大変。　あまり合理的ではない(力技)



解決策...

解決策1

一つ一つの歯に分けてコライダーを割り当て、
その小さな部品の集合を一つのオブジェクトとして扱う。

→処理が大変。　あまり合理的ではない(力技)

解決策2

すべてに動力を持たせるが、ただのアニメーションだと、、、



解決策...

解決策1

一つ一つの歯に分けてコライダーを割り当て、
その小さな部品の集合を一つのオブジェクトとして扱う。

→処理が大変。　あまり合理的ではない(力技)



解決策2

すべてに動力を持たせるが、ただのアニメーションだと、、、

…なので、親の歯車の回転速度をある変数でおいて、
子の歯車の回転速度をその変数によって変化させ、
あたかもかみ合っているように見せる。

解決策...

解決策1

一つ一つの歯に分けてコライダーを割り当て、
その小さな部品の集合を一つのオブジェクトとして扱う。

→処理が大変。　あまり合理的ではない(力技)



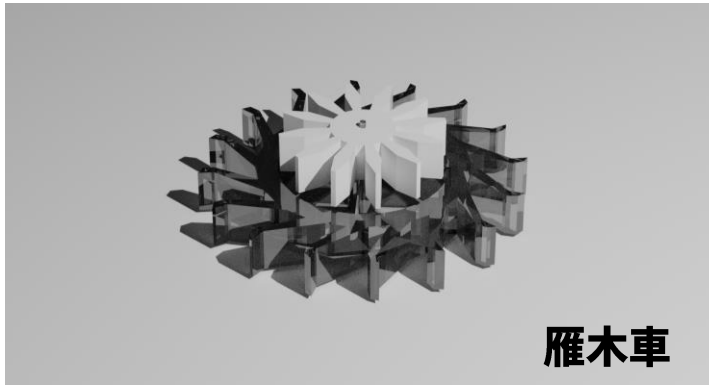
解決策2

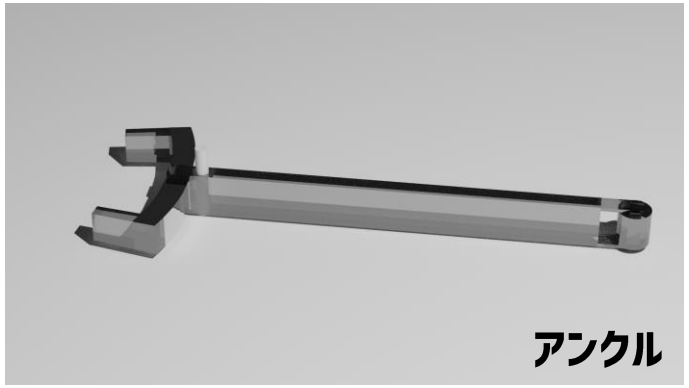
すべてに動力を持たせるが、ただのアニメーションだと、、、

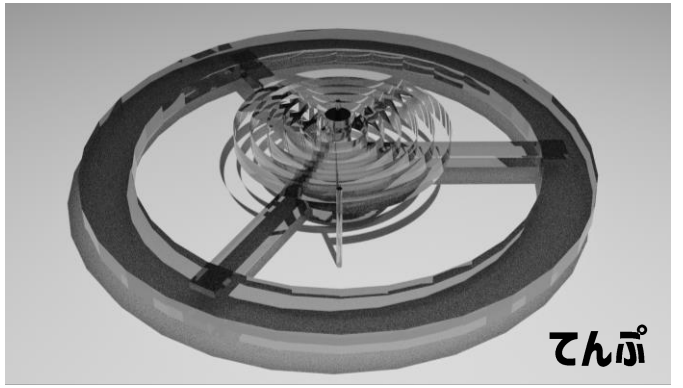
…なので、親の歯車の回転速度をある変数でおいて、
子の歯車の回転速度をその変数によって変化させ、
あたかもかみ合っているように見せる。

→物理演算というUnityの醍醐味は損なわれるが、
　C#で制御できるというUnityの強みを生かした策なのでは、、、？

設計の方針







アンクルにかみ合うような速度cを定める

アンクルのリズムと合うように(かみ合うように)
一方向に回るようなコードを書く

この時に速度cを用いて表す

てんぷにかみ合うような速度bを定める

てんぷにかみ合うような、
時計回り・反時計回りに振れるコードを書く

この時に速度bを用いて表す

速度aを定める

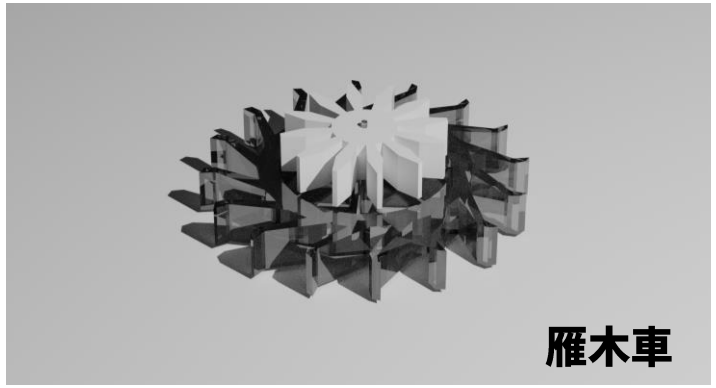
時計回り、反時計回りに振れるような
コードを書く

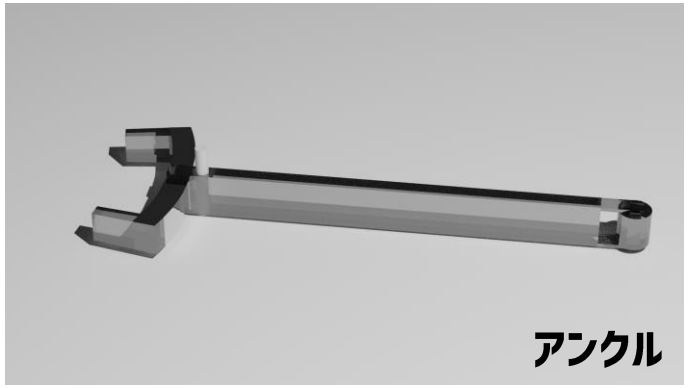
この時に速度aを用いて表す

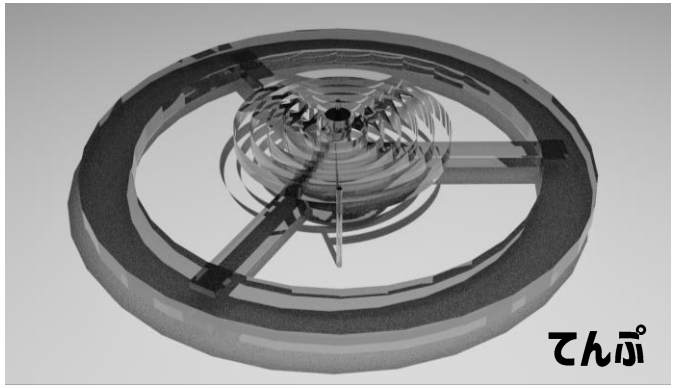


※イメージ

設計の方針





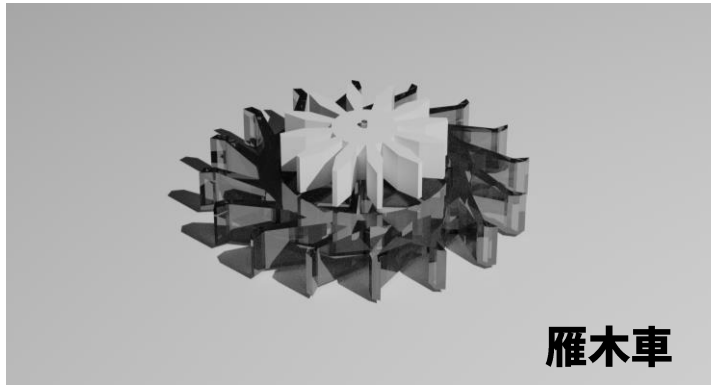


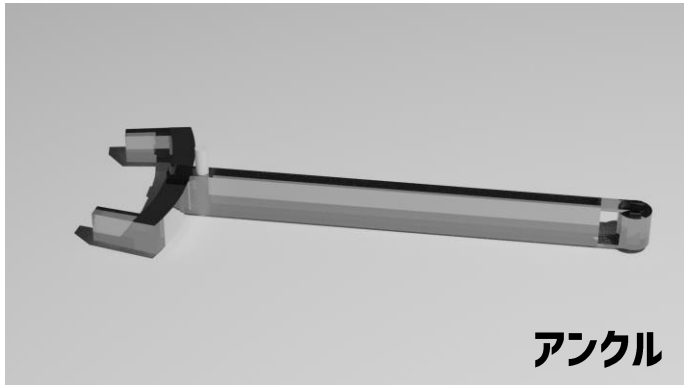
それぞれのかみ合う速度 a, b, c で表した各動作を $f(a), g(b), h(c)$とおくと、
$a : b : c$ が成り立っていれば、各動作$f(a), g(b), h(c)$は適切にかみ合う動きになる。

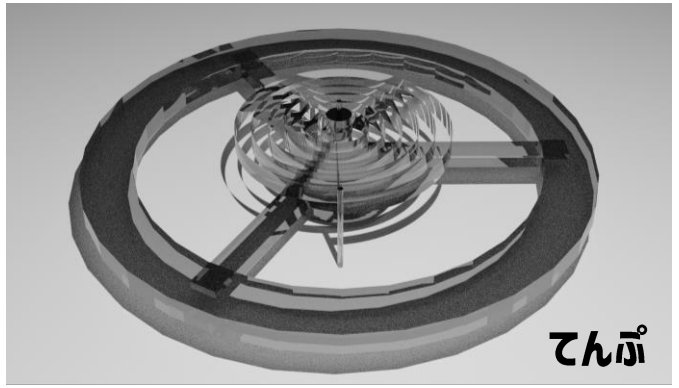


※イメージ

設計の方針







c
$h(c)$

b
$g(b)$

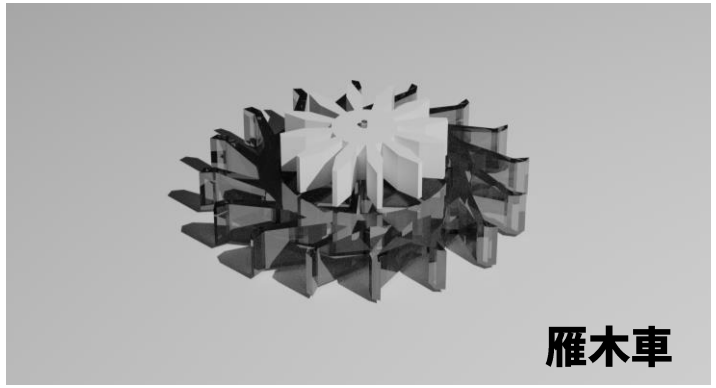
a
$f(a)$

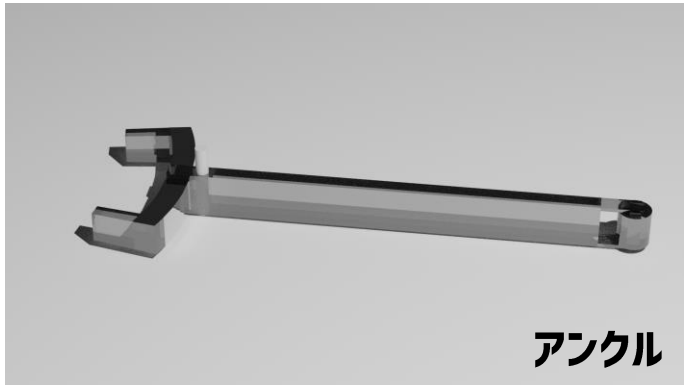
それぞれのかみ合う速度 a, b, c で表した各動作を $f(a), g(b), h(c)$とおくと、
$a: b: c$ が成り立っていれば、各動作$f(a), g(b), h(c)$は適切にかみ合う動きになる。

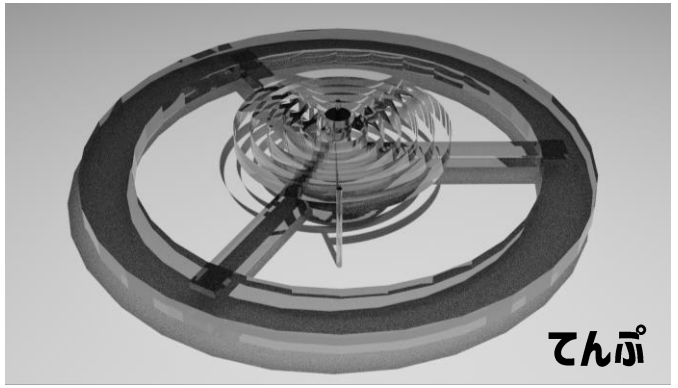


※イメージ

設計の方針







c
$h(c)$

b
$g(b)$

a
$f(a)$

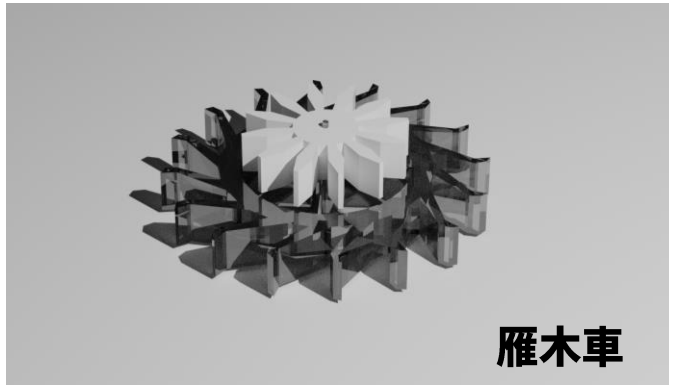
それぞれのかみ合う速度 a, b, c で表した各動作を $f(a), g(b), h(c)$とおくと、
$a: b: c$ が成り立っていれば、各動作$f(a), g(b), h(c)$は適切にかみ合う動きになる。

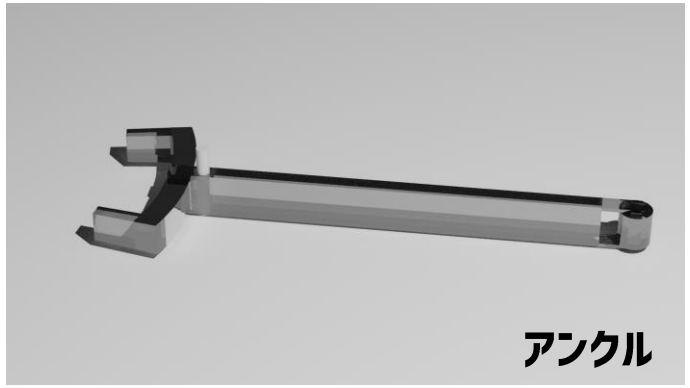
つまり、親のてんぷの速度aを変数で定めて、アンクル、雁木車の速度b, cをてんぷの速度aから参照して
適切な動作を行えるようなプログラムを書くとよい。

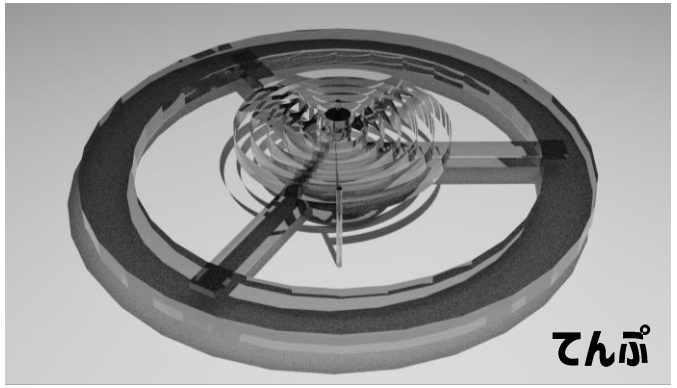


※イメージ

C#での実装







C# / 雁木車.cs

C# / あんくる.cs

C# / てんぷ.cs

```
float speedA = 0.1f;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
       (0f,speedA * i,0f,Space.World);

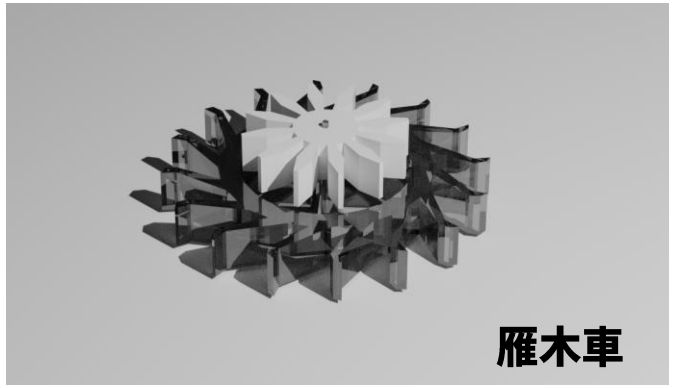
    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

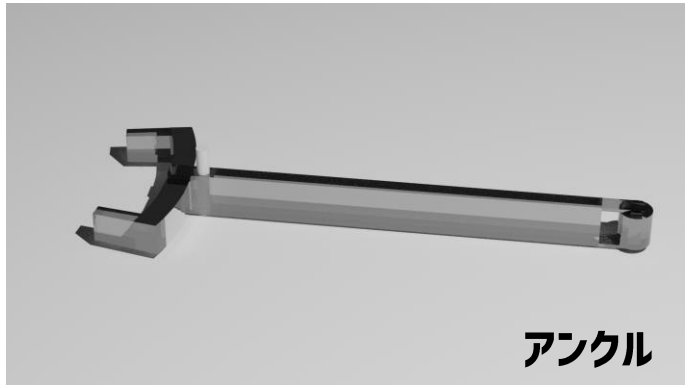
cnt += 1;
}
```

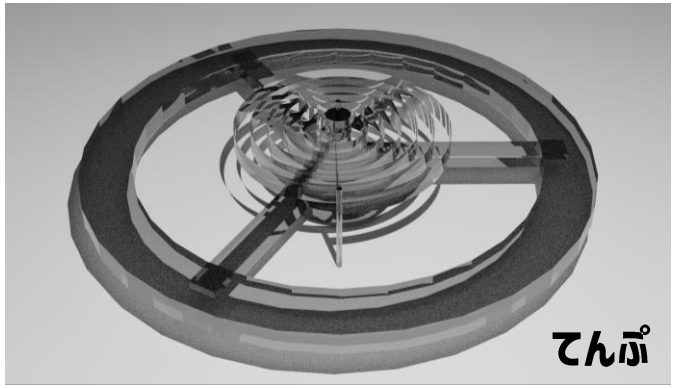


※イメージ

C#での実装







C# / 雁木車.cs

C# / あんくる.cs

```
float speedB = ～*speedA;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
     (0f,speedB/4 * i,0f,Space.World);

    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

cnt += 1;
}
```

C# / てんぷ.cs

```
float speedA = 0.1f;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
       (0f,speedA * i,0f,Space.World);

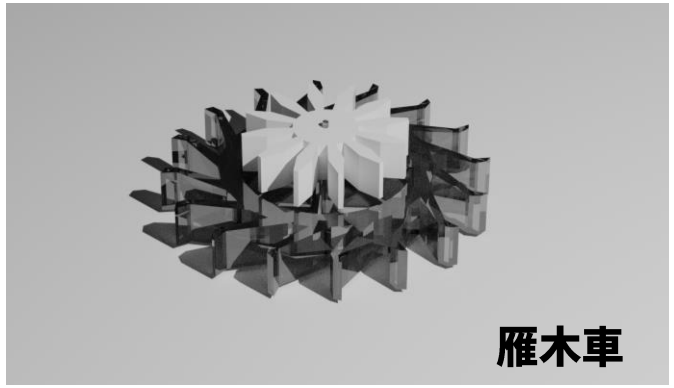
    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

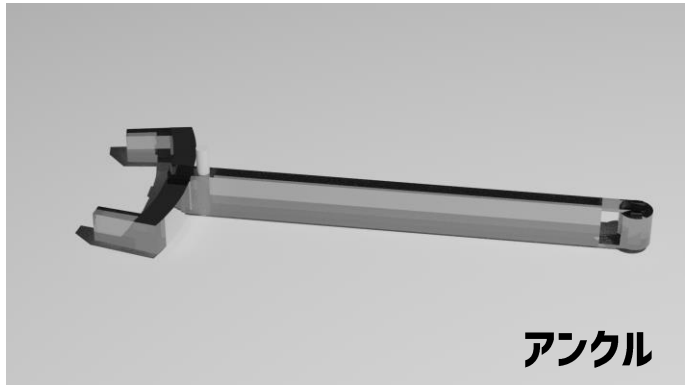
cnt += 1;
}
```

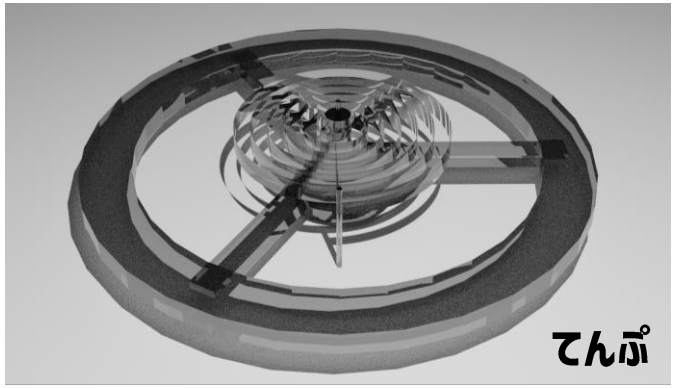


※イメージ

C#での実装







C# / 雁木車.cs

```
float speedC = ～*speedA;
int cnt = 1;
int i = 1;

void Update(){
    if(cnt%600 < 100){
        i = ～;
    }else if(cnt%600 <200){
        i = ～;
    }else if ～...
    }
    myTransform.Rotate
       (0f,speedC * i,0f,Space.World);

cnt += 1;
}
```

C# / あんくる.cs

```
float speedB = ～*speedA;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
     (0f,speedB/4 * i,0f,Space.World);

    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

cnt += 1;
}
```

C# / てんぷ.cs

```
float speedA = 0.1f;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
       (0f,speedA * i,0f,Space.World);

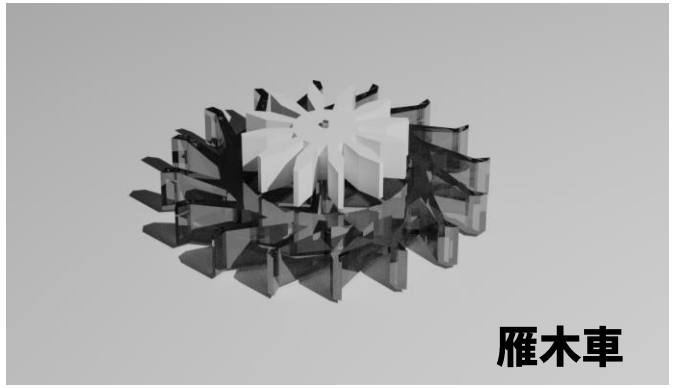
    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

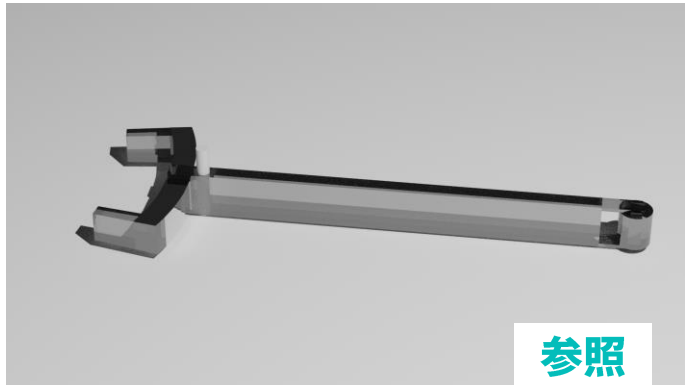
cnt += 1;
}
```

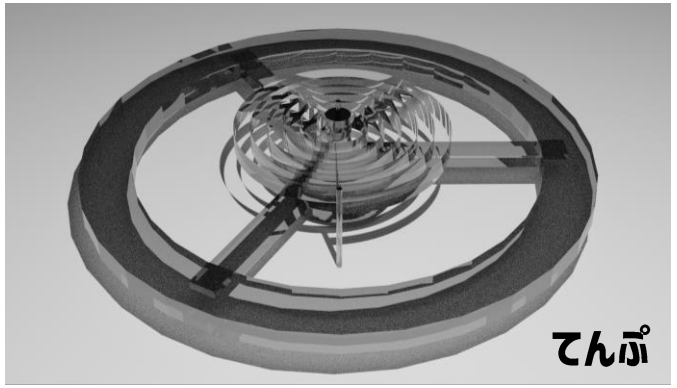


※イメージ

C#での実装







C# / 雁木車.cs

```
float speedC = ～*speedA;
int cnt = 1;
int i = 1;

void Update(){
    if(cnt%600 < 100){
        i = ～;
    }else if(cnt%600 <200){
        i = ～;
    }else if ～...
    }
    myTransform.Rotate
       (0f,speedC * i,0f,Space.World);

cnt += 1;
}
```

C# / あんくる.cs

```
float speedB = ～*speedA;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
     (0f,speedB/4 * i,0f,Space.World);

    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

cnt += 1;
}
```

C# / てんぷ.cs

```
float speedA = 0.1f;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
       (0f,speedA * i,0f,Space.World);

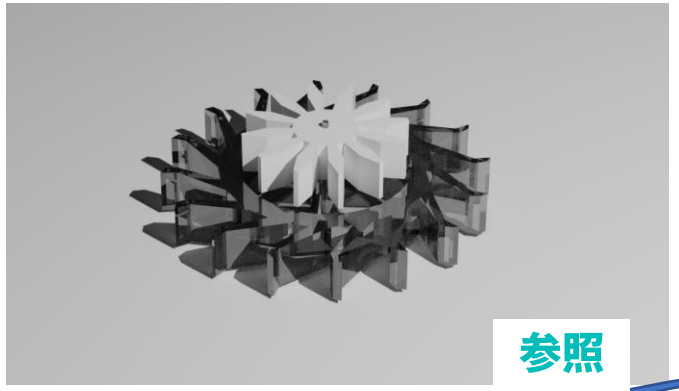
    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

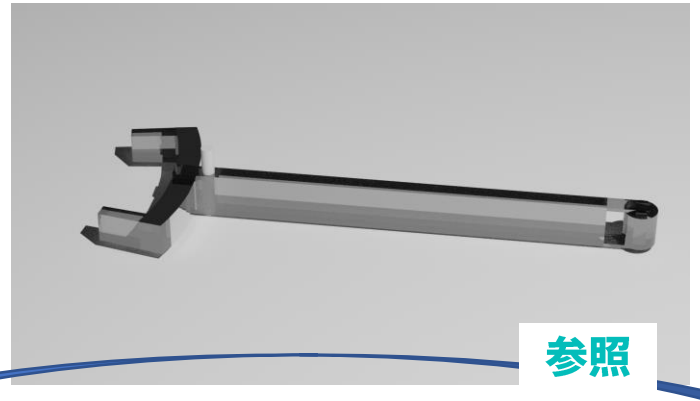
cnt += 1;
}
```

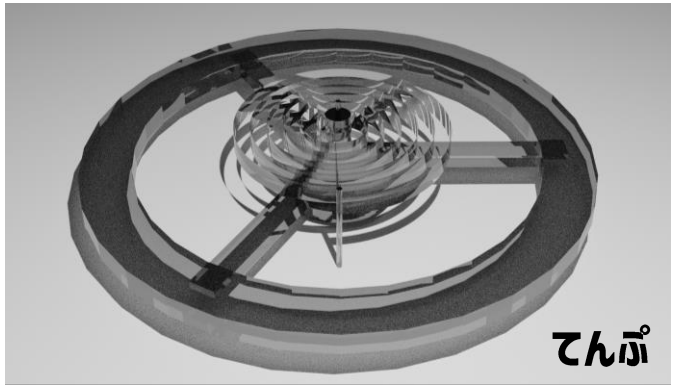


※イメージ

C#での実装







C# / 雁木車.cs

```
float speedC = ～*speedA;
int cnt = 1;
int i = 1;

void Update(){
    if(cnt%600 < 100){
        i = ～;
    }else if(cnt%600 <200){
        i = ～;
    }else if ～...
    }
    myTransform.Rotate
       (0f,speedC * i,0f,Space.World);

cnt += 1;
}
```

C# / あんくる.cs

```
float speedB = ～*speedA;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
     (0f,speedB/4 * i,0f,Space.World);

    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

cnt += 1;
}
```

C# / てんぷ.cs

```
float speedA = 0.1f;
int cnt = 1;
int i = 1;

void Update(){

    myTransform.Rotate
       (0f,speedA * i,0f,Space.World);

    if (cnt >= 600 || r <= 0){
        i *= -1;
    }

cnt += 1;
}
```



※イメージ

実際の動作

